ECE-9413 Final Report: Negacyclic NTT and SumCheck Prover

Koshiq Hossain (kh3134)

Jack Chou (cc9171)
New York University
ECE-9413

Abstract—This report describes two exact finite-field kernels implemented in JAX: a negacyclic number theoretic transform (NTT) and a SumCheck prover. Both submitted implementations preserve the provided public APIs and pass the required correctness tests. The NTT implementation uses a twisted Cooley–Tukey schedule with Montgomery arithmetic and a GPU-oriented Pallas path, while the SumCheck implementation uses a general expression evaluator, serialized per-round folding, overflow-safe 32-bit arithmetic, and an optional 64-bit core track. On the A100 GPU, the NTT reached 5.7 Gcoeff/s at $N = 65536$ and remained stable up to $N = 2^{24}$. The required 32-bit SumCheck prover reached roughly 0.2–0.3 ms for the linear vars20 expression and roughly 0.5–0.6 ms for the cubic vars20 expression. The most useful failed experiment was replacing reshape-based NTT stages with lax.fori_loop: compile time improved, but runtime regressed because the implementation lost contiguous reshape-based memory access. The main SumCheck optimization is not a protocol change; it is a cleaner JAX/XLA execution path with fewer integer-width conversions and less unused table dataflow.

I. Introduction

NTT and SumCheck are both central kernels in modern cryptographic proof and polynomial computation systems. They differ from ordinary dense floating-point GPU workloads because all arithmetic is exact modular integer arithmetic, where overflow behavior and reduction cost are part of the algorithm. The NTT is used to accelerate polynomial multiplication over finite fields. SumCheck reduces a large Boolean-hypercube sum to a sequence of low-degree univariate polynomial claims, making it a key primitive in proof systems.

The project contributions are: (1) a batched negacyclic NTT using a twist-plus-cyclic NTT schedule and Montgomery reduction, (2) a required 32-bit SumCheck prover for the base expressions, (3) overflow-safe modular arithmetic for both required 32-bit and optional 64-bit SumCheck tracks, and (4) benchmark evidence tying correctness and performance claims to the submitted code. All reported performance commands were run on an NVIDIA A100-SXM4-40GB GPU through the NYU HPC Slurm environment, with local CPU measurements retained as a baseline where useful.

II. Assignment 1: Negacyclic NTT

A. Design and Optimization

The implemented NTT evaluates

$$y[k] = \sum_{n=0}^{N-1} x[n]\psi^{(2k+1)n} \pmod{q}. \tag{1}$$

The code uses the standard negacyclic decomposition: first multiply each input coefficient by the appropriate power of ψ, then run a cyclic Cooley–Tukey NTT. This structure fits the provided tables and keeps the hot path regular across the batch dimension.

The modular arithmetic hot path uses unsigned 32-bit values, with widening only where multiplication requires it. The optimized path converts values into Montgomery form in prepare_tables. The precomputed tables include a pretwisted ψ table and Montgomery-form stage twiddles. The default path uses JAX reshapes and vectorized Gentleman–Sande DIF butterfly stages, which run on both CPU and GPU backends. Reshape isolates contiguous butterfly halves without explicit gather/scatter index arrays, allowing XLA to fuse the elementwise add, subtract, Montgomery multiply, compare, and select operations. An experimental Pallas kernel is kept opt-in, but the final A100 measurements use the stable JAX path. This keeps the public API unchanged while moving table preparation out of the timed benchmark region.

The most useful optimization was moving expensive table conversion and Montgomery constants into prepare_tables. A less attractive approach would have been direct evaluation, which is simpler but has $O(N^2)$ work and does not scale. Another attempted optimization used jax.lax.fori_loop to make the stage loop appear more compact to XLA. It reduced compile time on small RTX experiments, but runtime roughly doubled because dynamic loop shapes forced an index/gather implementation instead of the zero-copy reshape pattern. The final implementation therefore stays with static reshape-based $O(N \log N)$ butterflies.

B. Correctness

Table I lists the correctness commands required by the guidelines. The benchmark test wrapper must be run with uv –directory assignment1 from the repository root so its internal subprocess starts in the assignment directory.

TABLE I
Assignment 1 correctness checks

Command	Result
uv run pytest via submission script	5 passed
pytest –logn 10 –batch 4	5 passed
python -m tests.benchmark –tests –logn 10 –batch 4	5 passed

TABLE II
Assignment 1 A100 scaling sweep, batch 1

$\log_2(N)$	N	Compile (ms)	Median (ms)	Mcoeff/s
10	1024	709	0.081	50
16	65536	1700	0.734	5711
20	1048576	2346	6.58	5100
24	16777216	7031	37.0	3625

C. Performance

Table II reports the A100 sweep from the NTT presentation experiments. Throughput peaks near $N = 65536$, where the transform has enough work to amortize launch overhead while still fitting well in the memory hierarchy. At $N = 2^{24}$, throughput drops, suggesting that HBM traffic and L2 capacity become more important than launch overhead.

III. Assignment 2: SumCheck

A. Design and Optimization

The SumCheck prover supports expressions represented as lists of additive terms, where each term is a list of multiplicative factors. This general evaluator supports the four required base expressions without hard-coding a separate kernel per expression. For an expression of degree d, each round emits $d+1$ evaluations of the round polynomial. Thus the linear expression a emits two values per round, a*b and a*b + c emit three, and a*b*c emits four.

The implementation strictly follows the required round order. For each round, it first computes the round polynomial evaluations using the current tables. Only after that round does it fold the tables using the provided challenge. The code does not precompute challenge-folded tables ahead of time.

For the required 32-bit track, multiplication still widens to 64 bits because a 32-bit product can exceed 32 bits. Addition and subtraction, however, now avoid unnecessary 64-bit casts. mod_add_32 detects uint32 wraparound and corrects by adding $2^{32} \bmod q$ before the final conditional subtraction. mod_sub_32 computes the wrapped case as $q-(b-a)$, avoiding a+q overflow. This optimization keeps correctness on the edge cases while reducing the integer width used by common MLE and expression operations.

The implementation also reduces unused dataflow. The working dictionary is filtered to the polynomial tables that appear in the current expression, so a carries only table a, and a*b carries only a and b. Expression accumulation starts from the first term instead of a zero vector, avoiding one redundant modular addition for each expression evaluation.

The optional 64-bit core required a different approach. Since JAX does not provide a native uint128 type, direct 64-bit multiplication can lose high product bits. The submitted optional path uses two-limb Montgomery multiplication with 32-bit limbs for mod_mul_64, and pairwise overflow-safe modular addition for 64-bit reductions. This made the optional core64 correctness suite pass, although the required report baseline remains the 32-bit track.

TABLE III
Assignment 2 required 32-bit correctness checks

Command	Result
pytest –bits 32 –num-vars 4	20/20 cases passed
pytest –bits 32 –num-vars 16	20/20 cases passed
pytest –bits 32 –num-vars 20	20/20 cases passed
bash scripts/make_submission.sh	91 passed

TABLE IV
Assignment 2 vars20 A100 benchmark, 32-bit

Expression	Compile (ms)	Median (ms)	Mpts/s
a	≈2083–2387	0.2–0.3	3330–4290
a*b	≈3792–4175	0.3–0.4	2420–3030
a*b + c	≈4832–5271	0.4–0.5	2080–2530
a*b*c	≈6299–6890	0.5–0.6	1890–2170

B. READMEs

The Assignment 2 README was helpful for understanding the expected API, the required expressions, and the exact correctness and benchmark commands. The sumcheck_intro.md walkthrough was especially useful for checking the round order and confirming that challenge-based MLE updates must occur between rounds rather than before the protocol begins. The custom-cases documentation was useful for understanding possible extra experiments, but the final required results here use the provided public cases.

C. Correctness

Table III reports the required 32-bit correctness commands. Each command passed all five public cases and all four base expressions for the selected variable count.

D. Performance

The required benchmark commands were run with –runs 8 –warmup 3 on an A100 GPU. Table IV summarizes the final vars20 runs using ranges from the five public cases. The latency ordering follows expression degree and operation count: a is fastest, then a*b, then the two-term quadratic a*b + c, then the cubic expression a*b*c. As degree grows, the prover must emit more evaluation points per round and perform more modular multiplications per table element.

TABLE V
Assignment 2 vars20 A100 advanced polynomials, 32-bit

Expression	Median (ms)	Mpts/s
a*a*b*b*c	0.7–0.8	1200–1390
a*b*c + d*e	0.5–0.6	1520–1800
a*b*c*g + d*e*g	0.7–0.8	1210–1390

TABLE VI
Assignment 2 vars20 compile-time comparison

Expression	Reference (ms)	Final (ms)	Improvement
a	2457	2173	12%
a*b	4604	3941	14%
a*b + c	5312	5059	5%
a*b*c	7368	6624	10%

Table V reports the optional 32-bit advanced polynomials on vars20. These are not required for the base benchmark, but they demonstrate that the general expression evaluator supports repeated factors and multiple additive terms without special-case kernels.

Table VI compares the final implementation against the teammate/reference implementation on the same A100 vars20 benchmark. Both implementations use the same SumCheck protocol and pass correctness. The compile-time improvement comes from implementation shape: fewer uint32-to-uint64 widening operations in add/sub, less unused table dataflow, and fewer redundant modular additions. Runtime remains close because both versions still scan and fold the same 2^{20}-entry tables.

The local CPU baseline is retained in Table VII. It shows the same expression ordering, but with much higher vars20 latency.

IV. Cross-Assignment Discussion

The main shared lesson is that modular arithmetic choices dominate both assignments. In the NTT, Montgomery form is effective because each butterfly does many modular multiplications with precomputed twiddles. In SumCheck, the required 32-bit path benefits more from carefully avoiding unnecessary widened addition and subtraction while still widening multiplication. The NTT is more structured: every stage has a regular butterfly layout and a predictable twiddle access pattern. SumCheck is more expression dependent: degree and term count directly change the amount of work per round.

Memory movement also differs. NTT repeatedly permutes and combines arrays, so the schedule and table layout matter. SumCheck repeatedly halves the tables by MLE update; the largest rounds dominate runtime because later rounds quickly become small. This also explains why vars4 benchmark numbers are mostly overhead, while vars20 makes the differences between expressions visible.

TABLE VII
Assignment 2 scaling by variable count, representative CPU medians

Expression	vars4 (ms)	vars16 (ms)	vars20 (ms)
a	0.005–0.006	0.13–0.16	1.05–1.09
a*b	0.007–0.011	0.36–0.39	3.47–3.70
a*b + c	0.007–0.008	0.45–0.54	6.01–6.41
a*b*c	0.007–0.008	0.54–0.59	7.98–8.45

V. AI Use Reflection

AI assistance was useful for quickly producing first-pass implementations, debugging edge cases, and designing benchmark commands. The most useful suggestion was to verify the optional 64-bit SumCheck path separately from the required 32-bit path; this exposed a broken 64-bit multiplication routine even though the required submission already passed. A plausible but poor suggestion was to implement 64-bit multiplication with a simple shift-and-add loop. That was correct but too slow for full vars16 and vars20 tests, so it was replaced with a two-limb Montgomery method. The best workflow was to keep AI suggestions tied to public tests and benchmark output rather than trusting code structure alone.

VI. Conclusion

Both assignments pass their required public tests with the submitted code. Assignment 1 uses a Montgomery reshape-based negacyclic NTT and reached 5.7 Gcoeff/s on the A100 at $N = 65536$. Assignment 2 uses a general SumCheck prover with serialized round folding and overflow-safe modular arithmetic; on the A100, its vars20 medians range from roughly 0.2–0.3 ms for a to roughly 0.5–0.6 ms for a*b*c. The most important NTT optimization was moving Montgomery conversion and table preparation outside the hot path. The most important SumCheck optimization was keeping 32-bit add/sub operations in uint32 while still handling overflow correctly. On the A100, the SumCheck implementation shows about 5–14% lower vars20 compile time than the reference implementation while preserving competitive runtime and correctness.